

GarantiFi Hackathon MVP Brief for Edson

GarantiFi — Hackathon MVP Brief

For: Edson (Lead Solana Developer) **From:** George (CEO) **Date:** April 2026 **Status:** This document supersedes all previous technical specs in the project folder.

This brief has three parts:

1. **Business requirements — English.** What the product must do. The contract.
2. **Requisitos de Negócio — Português.** Mesma coisa em português, para referência.
3. **Technical Annex — English only.** Architecture, accounts, instructions, tests, deployment. Guidance, not orders — if you find a better technical path to the same business outcome, take it and flag it at the next standup.

The business parts (1 and 2) are the contract. The annex (3) is how we'd build it — you'll know better than we do whether every detail holds up in Anchor/Solana/Bubblegum as of April 2026. If something in the annex contradicts what you know to be true, say so before assuming.

PART 1 — Business Requirements (English)

What we're building

GarantiFi is the first Solana protocol that lets someone borrow a stablecoin loan against a physical luxury asset — a Rolex, a gold chain, a painting — while a real-world custodian holds the asset in a physical vault. The protocol only releases the loan money after custody is confirmed on-chain. When the borrower repays, the lender's position grows. When they don't, the custodian can sell the asset to make the lender whole.

For the hackathon, we're building the happy path end-to-end on Devnet, with a Rolex as the demo asset and USDC as the loan currency.

The demo story — what the judge sees

Picture a 3-minute demo video. The judge watches this sequence:

1. **Two lenders deposit USDC into a vault.** The vault balance goes from zero to, say, \$15,000. Both lenders see their individual position in the vault.
2. **A borrower shows up with his Rolex.** On the web app he enters "Rolex Submariner, appraised at \$20,000" and requests a \$10,000 loan. The system creates an on-chain record of his loan request. **No money has moved yet.** The UI shows status: *Waiting for custody confirmation*.
3. **The custodian confirms custody.** In the video, this is George signing a transaction acting as the custodian. The on-chain status flips from *Pending* to *Active*. Only now can the loan be disbursed.
4. **The loan is disbursed.** \$10,000 USDC moves from the vault to the borrower's wallet. Vault balance drops. Active loan count goes up by one.
5. **The borrower repays a month later.** He sends back \$10,000 plus interest. The vault balance climbs back up — and now it's higher than before, because interest was added. Both lenders see their position is worth more than when they deposited. Loan status: *Repaid*.
6. **Show the test suite passing.** Green check marks across the board. Three seconds of screen time. This proves the code works, not just the demo.

Six moments. If all six work on Devnet, we're in the top tier of submissions.

What the user sees at every step

Two UI elements carry the product's maturity. They're the difference between "a script that works" and "a product that thinks."

A progress tracker on every loan — a visible stepper showing the four states: *Requested* → *Custody Confirmed* → *Disbursed* → *Repaid*. Each step has a timestamp and a transaction link. The current step pulses. When the borrower's loan sits at step 2 waiting for custody, the judge sees the pause — and the pause is the pitch. Funds

aren't moving because custody isn't confirmed. That's the point, visualized.

A documents panel attached to every loan — showing the paperwork that makes a real Brazilian loan enforceable: the appraisal report, the Cédula de Crédito Bancário (CCB), the custody receipt, the insurance certificate. In the hackathon demo these are placeholder PDFs, clearly labeled as such. In production they become IPFS-pinned, digitally signed, and hash-committed on-chain — the V2 roadmap spells out how. The point here is that our cNFT isn't an abstract token — it's the on-chain anchor for a full legal document trail, the same one a Brazilian fintech would assemble for any CCB loan today.

The one thing that makes us different

Everything above, except step 3, is standard DeFi lending. Anyone can clone that.

Step 3 is the product. The loan money cannot leave the vault until custody is confirmed on-chain by a designated signer. This is what makes GarantiFi the first real-world-asset lending protocol on Solana that isn't just a financial wrapper — it actually connects the blockchain to a physical vault.

Technically this is one validation line in one instruction. If that line ever disappears — even “temporarily” during refactoring — we don't have a pitch. Treat it as production code from day one.

What must be visible and working by May 7

- A public GitHub repo under the GarantiFi org with commits spanning the hackathon window (Apr 6 – May 11).
- A deployed Solana program on Devnet with a recorded program ID.
- A web app on Vercel where someone can complete steps 1–5 above end-to-end.
- A test suite that runs with one command and passes.
- Three short artifacts: recorded demo video, recorded pitch video, clean README.

What's not in scope

These are for later — after the hackathon, after funding, after audit. Not our problem by May 10:

- What happens when a borrower doesn't repay (the demo borrower always repays).
- Renewing or extending loans.
- Real-time price oracles (prices are hardcoded).
- Actual KYC (handled off-chain anyway).
- A marketplace for trading vault positions.
- Any asset type other than the demo Rolex.
- Mobile app.
- Mainnet.

Timeline

When	What	How I know it's done
Apr 25	Scaffold	Repo exists, <code>anchor build</code> works, program ID deployed to Devnet
Apr 27	Vault working	Two lenders can deposit USDC, transactions visible on Solana Explorer
May 1	Loan request working	Borrower creates a request, cNFT visible on Explorer, status is “pending custody”
May 3	Security gate working	Custodian confirms, disburse works. Disburse without confirm fails.
May 5	Full cycle working	Borrower repays, lender position is worth more than at deposit
May 7	Frontend working	Full flow runnable from the web app
May 8	Demo rehearsed	Video script agreed, no surprises left

If any milestone slips by more than 24 hours, WhatsApp me the day it slips — not at the next standup. We'll cut scope together. We won't cut quality and we won't miss the deadline.

What I need you to confirm before we start

1. You can commit roughly 22 hours per week for the next three weeks.
2. You've read this document and the technical annex, and flagged anything unclear.
3. You have a Solana dev environment running and Devnet SOL in your wallet.

Reply yes / yes / yes — or tell me what's missing.

PARTE 1 — Requisitos de Negócio (Português)

O que estamos construindo

GarantiFi é o primeiro protocolo na Solana que permite a alguém tomar um empréstimo em stablecoin dando como garantia um ativo físico de luxo — um Rolex, uma corrente de ouro, uma obra de arte — enquanto um custodiante do mundo real guarda o ativo em um cofre físico. O protocolo só libera o dinheiro do empréstimo depois que a custódia é confirmada on-chain. Quando o tomador paga, a posição do credor cresce. Quando não paga, o custodiante pode vender o ativo para cobrir o credor.

Para o hackathon, estamos construindo o caminho feliz de ponta a ponta na Devnet, usando um Rolex como ativo de demonstração e USDC como moeda de empréstimo.

A história da demo — o que o jurado vê

Imagine um vídeo de 3 minutos. O jurado assiste a esta sequência:

1. **Dois credores depositam USDC em um cofre.** O saldo do cofre sai de zero e vai para, digamos, \$15.000. Os dois credores veem sua posição individual no cofre.
2. **Um tomador chega com o Rolex dele.** Na aplicação web, ele digita "Rolex Submariner, avaliado em \$20.000" e solicita um empréstimo de \$10.000. O sistema cria um registro on-chain da solicitação. **Nenhum dinheiro se moveu ainda.** A interface mostra o status: *Aguardando confirmação de custódia*.
3. **O custodiante confirma a custódia.** No vídeo, esse é o George assinando uma transação como custodiante. O status on-chain muda de *Pendente* para *Ativo*. Só agora o empréstimo pode ser desembolsado.
4. **O empréstimo é desembolsado.** \$10.000 USDC saem do cofre para a carteira do tomador. O saldo do cofre cai. O número de empréstimos ativos sobe em um.
5. **O tomador paga um mês depois.** Ele devolve \$10.000 mais juros. O saldo do cofre volta a subir — e agora está mais alto que antes, porque os juros foram somados. Os dois credores veem que a posição deles vale mais do que quando depositaram. Status do empréstimo: *Quitado*.
6. **A suíte de testes passando.** Tudo verde. Três segundos de vídeo. Isso prova que o código funciona, não só a demo.

Seis momentos. Se os seis funcionarem na Devnet, estamos na elite das submissões.

O que o usuário vê em cada etapa

Dois elementos de UI carregam a maturidade do produto. São a diferença entre "um script que funciona" e "um produto que pensa."

Um indicador de progresso em cada empréstimo — uma linha do tempo visível mostrando os quatro estados: *Solicitado* → *Custódia Confirmada* → *Desembolsado* → *Quitado*. Cada etapa tem timestamp e link da transação. A etapa atual pulsa. Quando o empréstimo do tomador fica parado na etapa 2 esperando a custódia, o jurado vê a pausa — e a pausa é o pitch. O dinheiro não se move porque a custódia não foi confirmada. Esse é o ponto, visualizado.

Um painel de documentos anexado a cada empréstimo — mostrando a papelada que torna um empréstimo brasileiro real e executável: laudo de avaliação, Cédula de Crédito Bancário (CCB), termo de custódia, apólice de seguro. Na demo do hackathon esses são PDFs placeholder, claramente identificados como tal. Em produção, viram IPFS-pinned, assinados digitalmente, com hash registrado on-chain — o roadmap V2 detalha como. O ponto aqui é que o nosso cNFT não é um token abstrato — é a âncora on-chain de uma trilha documental legal completa, a mesma que uma fintech brasileira montaria para qualquer empréstimo CCB hoje.

A única coisa que nos diferencia

Tudo acima, menos o passo 3, é empréstimo DeFi padrão. Qualquer um consegue clonar.

O passo 3 é o produto. O dinheiro do empréstimo não pode sair do cofre até que a custódia seja confirmada on-chain por um signatário designado. É isso que faz da GarantiFi o primeiro protocolo de empréstimo com ativos do mundo real na Solana que não é só um invólucro financeiro — ele de fato conecta a blockchain a um cofre físico.

Tecnicamente, é uma linha de validação em uma instrução. Se essa linha sumir — mesmo “temporariamente” durante uma refatoração — não temos pitch. Trate ela como código de produção desde o primeiro dia.

O que precisa estar visível e funcionando até 7 de maio

- Um repositório público no GitHub, dentro da organização GarantiFi, com commits dentro da janela do hackathon (6 de abril a 11 de maio).
- Um programa Solana deployado na Devnet, com o program ID registrado.
- Uma aplicação web na Vercel onde alguém consegue completar os passos 1 a 5 acima de ponta a ponta.
- Uma suíte de testes que roda com um comando e passa.
- Três entregáveis curtos: vídeo da demo gravado, vídeo do pitch gravado, README limpo.

O que não está no escopo

- Essas coisas ficam para depois — depois do hackathon, depois da captação, depois da auditoria. Não é problema nosso até 10 de maio:
- O que acontece quando um tomador não paga (o tomador da demo sempre paga).
 - Renovar ou estender empréstimos.
 - Oráculos de preço em tempo real (preços hardcoded).
 - KYC real (é tratado off-chain de qualquer forma).
 - Mercado secundário para posições do cofre.
 - Qualquer ativo que não seja o Rolex da demo.
 - App mobile.
 - Mainnet.

Cronograma

Quando	O quê	Como eu sei que está pronto
25 abr	Scaffold	Repo existe, <code>anchor build</code> funciona, program ID deployado na Devnet
27 abr	Cofre funcionando	Dois credores conseguem depositar USDC, transações visíveis no Solana Explorer
1 mai	Solicitação de empréstimo	Tomador cria solicitação, cNFT visível no Explorer, status “aguardando custódia”
3 mai	Gate de segurança funcionando	Custodiante confirma, desembolso funciona. Desembolso sem confirmação falha.
5 mai	Ciclo completo	Tomador paga, posição do credor vale mais que no depósito
7 mai	Frontend funcionando	Fluxo completo executável pela web app
8 mai	Demo ensaiada	Roteiro do vídeo acordado, sem surpresas

Se algum marco atrasar mais de 24h, me manda um WhatsApp no dia em que atrasar — não no próximo standup. A gente corta escopo junto. Não cortamos qualidade e não perdemos deadline.

O que preciso que você confirme antes de começarmos

1. Você consegue dedicar cerca de 22 horas por semana nas próximas três semanas.
2. Você leu este documento e o anexo técnico, e sinalizou qualquer coisa que não ficou clara.
3. Você tem um ambiente de dev Solana rodando e SOL de Devnet na carteira.

Responde sim / sim / sim — ou me diz o que está faltando.

PART 2 — Technical Annex (English)

Everything below is guidance. If anything contradicts what you know to be true about Anchor, Solana, or Bubblegum v2 in April 2026, raise it at the next standup before assuming. Don't guess.

A1. Scope — 5 instructions

Exactly five on-chain instructions. Nothing more for the hackathon.

#	Instruction	Purpose
1	<code>initialize_vault</code>	Create the lender pool with parameters (min deposit, max LTV, target APY, stablecoin mint).
2	<code>deposit_capital</code>	Lender deposits stablecoin, receives vault shares pro-rata.
3	<code>create_ccb_trdc</code>	Borrower opens a loan request. cNFT minted via Bubblegum v2. Status = <code>PendingCustody</code> . No funds move.
4a	<code>confirm_custody</code>	Custodian signs → status flips to <code>Active</code> . This is the security gate.
4b	<code>disburse_ccb</code>	Requires <code>status == Active</code> . CPI transfer of principal to borrower.
5	<code>repay_ccb</code>	Borrower repays principal + interest. Vault <code>total_deposited</code> grows by interest. Lender shares gain value pro-rata.

A2. Tech stack

- **Framework:** Anchor 0.30 (latest stable).
- **Languages:** Rust on-chain, TypeScript for tests and frontend.
- **cNFT library:** Metaplex Bubblegum v2. Pre-create one Merkle tree at deployment; it hosts all loans.
- **Stablecoin (demo):** Devnet USDC. BRZ on Devnet is flaky — BRZ integration is roadmap narrative, not hackathon scope.
- **Storage:** on-chain account data + off-chain metadata URI. Use a mock IPFS hash for the demo; don't spend time on real IPFS pinning.
- **Frontend:** Next.js + Tailwind + @solana/wallet-adapter + Anchor client. Deploy to Vercel.
- **Testing:** Anchor TypeScript tests.

A3. Account architecture

Vault (PDA) — seeds `[b"vault", vault_id.to_le_bytes()]`:

```
pub struct Vault {
  pub vault_id: u64,
  pub authority: Pubkey,
  pub stablecoin_mint: Pubkey,
  pub treasury: Pubkey,           // SPL token account owned by vault PDA
  pub total_deposited: u64,
  pub total_shares: u64,
  pub active_loans_value: u64,
  pub min_deposit: u64,
  pub target_apy_bps: u16,
  pub max_ltv_bps: u16,           // hard cap 7500 (75%)
  pub withdrawal_cooldown: i64,
  pub created_at: i64,
  pub bump: u8,
}
```

LenderPosition (PDA) — seeds `[b"position", vault.key().as_ref(), lender.key().as_ref()]`:

```
pub struct LenderPosition {
    pub vault: Pubkey,
    pub lender: Pubkey,
    pub shares: u64,
    pub total_deposited: u64,
    pub last_deposit_at: i64,
    pub bump: u8,
}
```

Trdc — the loan / custody record (PDA) — seeds `[b"trdc", borrower.key().as_ref(), trdc_nonce.to_le_bytes()]`:

```
pub struct Trdc {
    pub vault: Pubkey,
    pub borrower: Pubkey,
    pub custody_node: Pubkey,          // hardcoded signer in demo
    pub appraised_value: u64,
    pub principal: u64,
    pub interest_rate_bps: u16,        // monthly
    pub ltv_bps: u16,
    pub maturity_ts: i64,
    pub status: TrdcStatus,
    pub origination_ts: i64,
    pub asset_uri: String,             // mock IPFS hash OK
    pub cnft_asset_id: Pubkey,
    pub bump: u8,
}

pub enum TrdcStatus {
    PendingCustody,
    Active,
    Repaid,
    Defaulted, // enum member; never set in hackathon
}
```

Treasury — standard SPL associated token account owned by the Vault PDA. **Bubblegum tree** — pre-created once per deployment, hosts all loans.

A4. Instruction details

A4.1 initialize_vault

```
pub fn initialize_vault(
    ctx: Context<InitializeVault>,
    vault_id: u64,
    stablecoin_mint: Pubkey,
    min_deposit: u64,
    target_apy_bps: u16,
    max_ltv_bps: u16,
    withdrawal_cooldown: i64,
) -> Result<()>
```

Validation: `max_ltv_bps <= 7500, target_apy_bps <= 5000, min_deposit > 0.`

A4.2 deposit_capital(amount: u64)

1. `amount >= vault.min_deposit`
2. Shares: if `total_shares == 0` then `shares = amount`, else `shares = amount * total_shares / total_deposited`
3. CPI transfer from lender → `vault_treasury`
4. Update vault counters
5. Init or update `LenderPosition`
6. Emit `Deposited { vault, lender, amount, shares }`

A4.3 create_ccb_trdc(appraised_value, ltv_bps, principal, interest_rate_bps, maturity_ts, asset_uri)

Validation: - `ltv_bps <= vault.max_ltv_bps - principal == appraised_value * ltv_bps / 10_000` (exact match, not just $\leq$) - `maturity_ts > now + 30 days`

Logic: 1. Mint cNFT via Bubblegum v2 CPI. Metadata { name: "TRDC #nonce", symbol: "TRDC", uri: asset_uri }. **Leaf owner = vault PDA, NOT borrower.** The custody claim belongs to the protocol. 2. Init Trdc with status = PendingCustody 3. Emit TrdcCreated { trdc, cnft_asset_id, borrower, principal }

No funds move in this instruction. Only the cNFT mint and loan record creation.

A4.4 confirm_custody — THE SECURITY GATE

Accounts: trdc (mut), custody_node (signer, must equal trdc.custody_node).

1. Verify trdc.status == PendingCustody (else error).
2. Verify signer pubkey == trdc.custody_node.
3. Set trdc.status = Active.
4. Emit CustodyConfirmed { trdc, custody_node, timestamp }.

In the demo, custody_node is a hardcoded keypair George controls — he signs this call in the demo video acting as the Gitel custodian.

A4.5 disburse_ccb — the payout (and the invariant)

Validation (THE invariant): - trdc.status == Active (not PendingCustody, not Repaid, not Defaulted). - vault_treasury.amount >= trdc.principal.

Logic: 1. CPI transfer trdc.principal from vault_treasury → borrower_stablecoin. 2. vault.active_loans_value += trdc.principal. 3. trdc.origination_ts = Clock::get()?.unix_timestamp. 4. Emit Disbursed { trdc, borrower, principal, timestamp }.

Do not merge this with confirm_custody. The two-step separation is the security story. One instruction validates custody, another moves funds. Keep them apart.

A4.6 repay_ccb(repay_amount: u64)

1. trdc.status == Active
2. months_elapsed = (now - origination_ts) / SECONDS_PER_MONTH
3. interest = principal * interest_rate_bps * months_elapsed / 10_000
4. total_owed = principal + interest
5. repay_amount >= total_owed
6. CPI transfer from borrower → vault_treasury
7. Update vault: active_loans_value -= principal; total_deposited += interest (this is how lender shares gain value)
8. trdc.status = Repaid
9. Skip cNFT burn for the demo — Bubblegum burn is finicky and doesn't affect the narrative
10. Emit Repaid { trdc, borrower, principal, interest, timestamp }

A5. Test suite — tests/full_lifecycle.ts

Six tests, all green on anchor test:

1. **Happy path end-to-end.** Init vault → 2 lenders deposit → borrower creates TRDC → custody confirms → disburse → borrower repays → verify lender share value increased pro-rata.
2. **Disburse-without-custody blocked.** disburse_ccb called while status == PendingCustody must fail with CustodyNotConfirmed.
3. **Double-disburse blocked.** Second disburse_ccb after the first succeeds must fail.
4. **Share math.** Two lenders depositing different amounts at different times receive correct pro-rata shares.
5. **LTV cap.** create_ccb_trdc with ltv_bps > vault.max_ltv_bps fails.
6. **Insufficient treasury.** disburse_ccb when vault balance < principal fails.

A 3-second screenshot of the green test suite closes the demo video.

A6. Frontend — minimal React dApp

Three screens. Next.js + Tailwind + wallet adapter. Deploy to Vercel on app.[brand].com.

Lender screen: connect wallet (Phantom, Backpack, Solflare). Display: "Vault balance: X USDC / Your shares: Y (≈ Z USDC at current NAV)." Deposit form, submit, show tx signature link to Solana Explorer.

Borrower screen: connect wallet. Loan request form with asset dropdown (Rolex only for demo), appraised value

input, requested principal input, live-computed LTV, displayed interest rate. Submit mints cNFT and creates PendingCustody loan. “Your active loans” list with status badge and repay button per loan.

Public dashboard: total TVL across vaults, active loans count + total value, recent transactions (last 10, pulled from Anchor events).

Loan lifecycle stepper (shared component, ~4h). Renders on the Borrower screen (per loan) and Public dashboard (expanded per loan on click). Four fixed steps: Requested → Custody Confirmed → Disbursed → Repaid. Data source: `Trdc.status` + program event timestamps. Visual: horizontal dots connected by lines. Filled dot = completed, hollow = pending, pulsing ring = current step. Each completed step shows timestamp + Solana Explorer tx link. Use a copy-paste Tailwind/shadcn stepper pattern — do not design from scratch. 2-hour visual ceiling, 2-hour wiring.

Document cards (shared component, ~3h). Renders per loan. Four card tiles, in order:

1. Appraisal Report
2. Cédula de Crédito Bancário (CCB)
3. Custody Receipt — greyed-out until `trdc.status >= Active`
4. Insurance Certificate

Each card: filename, upload date (can be mocked as `origination_ts`), “View PDF” link. All four link to static sample PDFs served from Next.js `/public/docs/sample_*.pdf` — one sample PDF per document type, shared across all demo loans. Label the panel header: “Documents (demo — production will use IPFS + digital signatures).”

Hard constraint on document cards: no upload flow, no IPFS, no backend, no real pinning, no signed URLs, no hash commitments. Static assets only. If Edson finds himself writing upload handlers or installing IPFS libraries, stop and escalate. This is V2 work.

UI polish: clean, not pretty. Default Tailwind. Consistent spacing. No custom design. Judges care about functionality, not beauty.

A7. Deployment sequence

1. `anchor build` succeeds locally → `.so` file generated.
2. `anchor deploy --provider.cluster devnet` → record program ID.
3. Program ID pasted into frontend env + README.
4. Script: create + initialize a demo vault.
5. Pre-create Bubblegum Merkle tree.
6. Full lifecycle runs on Devnet (not just localnet).
7. Frontend deployed to Vercel.
8. Demo video recorded, MP4 in shared Drive.

A8. Emergency scope-cut ladder

If time gets tight, cut in this order. Never cut the top four.

Never cut: - The 5 instructions. - The custody-gate invariant. - Green tests for happy path + disburse-without-custody. - Devnet program ID + recorded transactions.

Cut in this order if needed: 1. Document cards (keep the stepper — it’s doing more narrative work per hour). 2. Public dashboard screen (keep lender + borrower + stepper). 3. Share-math + LTV-cap + insufficient-treasury tests (keep happy-path + security-gate only). 4. Frontend entirely — fall back to running the demo from an Anchor TS script with Solana Explorer links. 5. Last resort: hardcode flat 3% interest instead of time-based calc.

A9. Non-negotiables

- Code committed during the hackathon window (Apr 6 – May 11). Older code disqualifies the submission.
- Public GitHub repo under the GarantiFi org — not your personal account.
- MIT license.
- Devnet only. Mainnet comes after audit, post-accelerator.
- The custody-gate invariant (`disburse_ccb` requires `status == Active`) must hold at every commit. If it ever leaves the code “temporarily,” message George the same day — not at the next standup.

A10. Communication

- Standup every 2 days, 30 minutes. Come ready with: shipped / next / blocked.
 - Broken milestone = WhatsApp George the day it breaks.
 - Weekly GitHub activity must be visible — regular commits, not one massive dump at the end. Judges read commit history.
-

End of document. If anything here is unclear, ambiguous, or contradicts what you know to be true about Anchor/Solana/Bubblegum in April 2026, raise it at the next standup before assuming. Don't guess.